Laboratório Funcional · Aula 12

VITOR HUGO ASSIS DA SILVA · RA 241400232 · Turma B
Gerado em 16/10/2026, 20:09:40 · tempo registrado no cronômetro: 22:26

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora() é impura porque depende do relógio do sistema e pode devolver resultados diferentes sem receber parâmetros. registra(msg) é impura porque produz o efeito colateral de escrever na tela, em vez de apenas devolver um valor.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2

4: 14

7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
Execução original:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

Com compor(g, f):
1: A-1 A-2
```

```
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função altera e lê a variável global contador. Por isso, cada chamada muda o estado e produz A-1, A-2 etc., mesmo com a mesma entrada. Isso dificulta os testes porque o resultado depende da ordem das chamadas e exige reiniciar ou controlar o estado global.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = f(g(4)) = f(12) = 14. Já compor(g, f)(4) = g(f(4)) = g(6) = 18. Os resultados diferem porque a composição aplica as funções em ordens diferentes.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria e devolve uma lista nova, mantendo notas inalterada. notas.sort() altera a própria lista e devolve None. sorted segue melhor o estilo funcional porque evita mutação e produz um novo valor.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
Saída original:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

Sem o apóstrofo:
ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo impede a avaliação da lista e faz com que ela seja tratada como dado. Sem ele, o Scheme interpretou (A B C) como uma chamada da função A, com B e C como argumentos. Como A não estava definida, ocorreu o erro Unbound variable: A.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Em Scheme deve funcionar, porque a chamada é recursiva em cauda e o Scheme pode reaproveitar o mesmo registro de ativação. Em Python, a versão com um milhão de chamadas deve parar com RecursionError, pois Python não otimiza chamadas em cauda.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
...
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum cresce a expressão com as multiplicações pendentes e, consequentemente, a pilha de chamadas. Isso acontece porque cada chamada ainda precisa multiplicar seu resultado por n quando a chamada seguinte retornar. Na recursão em cauda, o resultado parcial vai no acumulador e nada fica pendente.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme otimiza chamadas em cauda e pode reutilizar o mesmo registro de ativação, mantendo espaço constante. Python não faz essa otimização: cada chamada cria um novo registro na pilha até atingir o limite de

recursão, causando RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else
         (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi preservar a ordem da lista em pares. Usei null? como caso-base em todas as funções, car para examinar o elemento atual e cdr para chamar a função recursivamente com o restante. Em pares, cons coloca o elemento atual no resultado quando ele é par.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime [0,1,4,9,16]. primeiro 42 (div 1 0) imprime 42, sem erro de divisão por zero.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Com member 17 squares: a execução não termina e acaba excedendo o limite de tempo.
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Haskell usa avaliação preguiçosa, portanto calcula somente os elementos da lista infinita exigidos por take. Em primeiro 42 (div 1 0), o segundo argumento não é utilizado pela função, então Haskell nunca avalia a divisão por zero.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 16 squares termina porque 16 aparece na lista. Já 17 nunca aparece, e a função continua procurando para sempre na lista infinita. Como a lista está ordenada, a busca pode devolver False assim que o elemento atual ultrapassar 17.

```
member b [] = False
member b (a:x)
 | a == b = True
 | a > b = False
 | otherwise = member b x
```

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15. [1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida forma um fechamento, ou closure. Ela mantém uma referência ao ambiente léxico em que foi criada, incluindo o valor n = 10, mesmo depois de somador(10) terminar.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

f.compose(g) calcula f(g(4)) = f(12) = 14. f.andThen(g) calcula g(f(4)) = g(6) = 18. f.compose(g) corresponde ao compor(f, g) da Prática 1.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
Exercício 1:
244

Exercício 2:
ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Depois de entender as operações, considero a versão funcional mais fácil de ler porque mostra diretamente as etapas de filtrar, transformar e reduzir. A versão com laço pode ser mais fácil de depurar passo a passo, pois permite observar diretamente o acumulador e os valores intermediários.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender porque declara diretamente a intenção: gerar os números, elevar ao cubo e somar. Eu aceitaria o custo em sistemas administrativos, educacionais ou de análise de dados, nos quais clareza e manutenção são mais importantes. Não aceitaria em sistemas de tempo real, dispositivos com poucos recursos ou aplicações de alto desempenho.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi uma linguagem conseguir trabalhar com listas infinitas calculando somente os elementos realmente necessários.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão é b <- h; o correto é b <= h. Em uma compreensão, <- significa retirar elementos de uma lista, então o compilador tratou h como uma lista e inferiu para a função o tipo incorreto sort :: [a] -> [[a]]. A comparação correta mantém os elementos menores ou iguais ao pivô e permite inferir Ord a => [a] -> [a].